ECE Paris – OMNES Education
Big Data Frameworks · ING4 Data & IA

Compte rendu – Lab Spark RDD

Introduction à l'API RDD de Spark

Partitions, évaluation paresseuse et lignage, limites d'un lecteur sans schéma, parsing manuel, reduceByKey et groupByKey, dédoublonnage et jointures sur des RDD clé-valeur, et exercices

Auteure	Dorline TIAKO OKASSIE
Groupe	ING4 – Groupe 01 – Data & Intelligence Artificielle

Environnement	Service Onyxia `vscode-pyspark` (4 CPU / 4000 Mi), Spark 4.1.1 en mode `local[*]` (4 cœurs), Python 3.13
Données	Copies locales de `bronze/users.csv` et `bronze/orders.csv` (bucket `s3://user-d-tiakookassie-ece`)
Dépôt Git	`github.com/Dorline0121/ece-2026-bigdata`
Date	5 novembre 2026

Sommaire

1. Objectifs du lab
2. Difficultés rencontrées et adaptations
3. Environnement et SparkContext
4. Création des RDD et partitions
5. Évaluation paresseuse et lignage
6. Le coût de l'absence de schéma : l'adresse sur deux lignes
7. Parsing manuel des commandes
8. RDD clé-valeur : reduceByKey et groupByKey
9. Dédoublonnage manuel
10. Jointure des commandes et des utilisateurs
11. Exercices
12. Synthèse et conclusion

1. Objectifs du lab

- Construire des RDD à partir des données bronze et observer leur partitionnement.
- Observer l'évaluation paresseuse : les transformations construisent un lignage, seules les actions l'exécutent.
- Comparer `reduceByKey` et `groupByKey` sur la même agrégation, en mesurant le shuffle.
- Dédoublonner les commandes à la main avec `reduceByKey`, équivalent RDD de la fonction de fenêtre du lab DataFrame.
- Joindre utilisateurs et commandes sous forme de RDD clé-valeur, et comprendre le coût d'une jointure.
- Comprendre ce que l'API DataFrame et Spark SQL apportent, en s'en passant d'abord.

2. Difficultés rencontrées et adaptations

> **① Blocs `for` et `def` dans le shell Python**
>
> **Constat.** Dans le shell interactif, une boucle `for` ou une fonction `def` doit être suivie d'une ligne vide. Collée avec d'autres instructions, elle provoque une `SyntaxError` (figure 3).
>
> **Adaptation.** Boucles remplacées par des expressions (`"\n".join(...)`) et fonctions écrites en `lambda`. La fonction `parse_order` conserve son comportement strict : elle échoue si une ligne n'a pas exactement 5 champs.

② Mesure du shuffle sans passer par le navigateur

Constat. Le sujet demande de comparer les volumes de shuffle dans la Spark UI, qui n'est accessible qu'en redirigeant le port 4040.

Adaptation. Les métriques ont été lues directement dans l'**API REST** de la Spark UI (`/api/v1/applications/…/stages` et `/storage/rdd`), depuis Python. Une petite fonction affiche les octets et enregistrements écrits par chaque nouveau stage de shuffle.

③ « Enregistrements » de shuffle en PySpark

Constat. Le nombre d'enregistrements de shuffle mesuré est 8 pour `reduceByKey` comme pour `groupByKey`. En PySpark, les données passent de Python à la JVM par **lots sérialisés** : Spark compte ces lots, pas les couples clé-valeur.

Adaptation. La comparaison s'appuie sur le **volume en octets**, qui reflète réellement la quantité de données échangées.

3. Environnement et SparkContext

Il n'existe pas de lecteur RDD pour S3A : les deux fichiers bronze sont téléchargés une fois en local. Aucune configuration S3 ni catalogue Iceberg n'est nécessaire, la session créée par le shell est donc conservée.

```
aws s3 --profile default cp "s3://$LAB_BUCKET_NAME/bronze/users.csv" users.csv
aws s3 --profile default cp "s3://$LAB_BUCKET_NAME/bronze/orders.csv" orders.csv
pyspark --master "local[*]"
>>> sc = spark.sparkContext
```

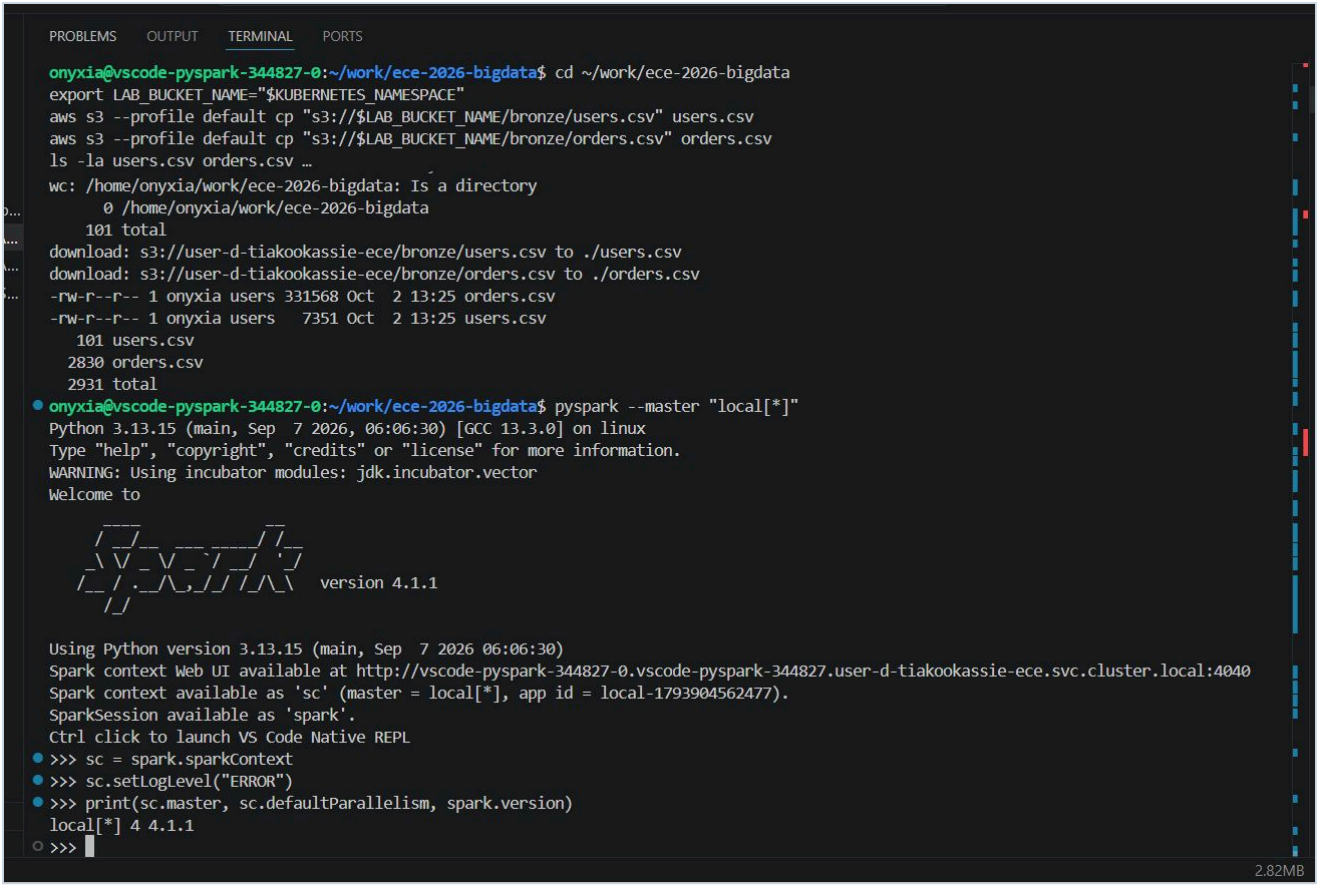


Figure 1 – Copie locale des CSV (101 et 2830 lignes), lancement de PySpark et SparkContext : local[*], 4 cœurs, Spark 4.1.1

`wc -l` compte **101 lignes** pour `users.csv` (1 en-tête et 50 utilisateurs sur 2 lignes chacun, à cause de l'adresse) et **2830** pour `orders.csv` (1 en-tête et 2829 commandes).

4. Création des RDD et partitions

```
users_lines = sc.textFile("users.csv")
orders_lines = sc.textFile("orders.csv")
orders_lines.getNumPartitions()                 # 2
orders_lines.repartition(8).getNumPartitions()  # 8
orders_lines.coalesce(1).getNumPartitions()     # 1
```



Figure 2 – Partitions par défaut (2 et 2), repartition(8) et coalesce(1), lignage du filtre et deux count()

`textFile()` lit le fichier **ligne par ligne** : chaque élément du RDD est une ligne de texte brute, sans en-tête reconnu, sans colonne et sans type.

Q1. Qu'est-ce qui détermine le nombre de partitions choisi par défaut par textFile(), pour un fichier de cette taille ?

`textFile(path, minPartitions)` utilise par défaut `minPartitions = min(defaultParallelism, 2)`, soit **2** ici (`defaultParallelism` vaut 4). Le fichier est ensuite découpé en *splits* Hadoop, au moins `minPartitions`, et davantage si le fichier dépasse la taille de bloc (32 Mo en local, 128 Mo sur HDFS). Avec 331 Ko et 7 Ko, les deux fichiers sont bien plus petits qu'un bloc : le minimum de 2 partitions s'applique. Un fichier de plusieurs Go en aurait eu une par bloc.

Q2. repartition() et coalesce() changent toutes deux le nombre de partitions. Laquelle déclenche un shuffle, laquelle non ?

`repartition(n)` fait un shuffle complet : toutes les lignes sont redistribuées uniformément entre les `n` partitions. Elle permet d'**augmenter** ou de diminuer leur nombre et produit des partitions équilibrées, au prix d'un échange de données. **`coalesce(n)` n'en fait pas** : elle regroupe localement des partitions existantes, sans échange réseau. Elle ne peut que **diminuer** le nombre de partitions, qui peuvent alors être déséquilibrées. `repartition(n)` n'est autre que `coalesce(n, shuffle=True)`.

5. Évaluation paresseuse et lignage

```
header = orders_lines.first()
orders_data = orders_lines.filter(lambda line: line != header and line.strip() != "")
print(orders_data.toDebugString().decode())
print(orders_data.count())   # 2829, à exécuter deux fois
```

Le *debug string* (figure 2) montre un **lignage**, pas un résultat : `PythonRDD[11]` (le filtre) repose sur `MapPartitionsRDD[3]`, lui-même sur `HadoopRDD[2]` qui lit `orders.csv`, en 2 partitions. À ce stade, aucune ligne n'a été lue : Spark n'a enregistré qu'une recette. Le calcul n'est lancé que par l'action `count()`, qui renvoie 2829.

Q3. Appelez orders_data.count() une seconde fois. La Spark UI montre-t-elle un nouveau stage ? Pourquoi ?

Oui : chaque action lance un **nouveau job**, avec son propre stage. Un RDD ne stocke pas de données, seulement son lignage. Au second `count()`, Spark réexécute donc toute la recette : il relit `orders.csv`, refait le filtre et recompte. Le résultat est identique (2829), mais le travail est entièrement refait.

Q4. À quel moment de ce notebook .cache() changerait-il cette réponse ?

Il faudrait appeler `.cache()` sur `orders_data` **avant la première action**, juste après le `filter`. Le premier `count()` calculerait alors le RDD et le garderait en mémoire ; le second créerait toujours un job, mais lirait les partitions depuis le cache, sans relire le fichier ni refaire le filtre. Un `.cache()` placé après les deux `count()` ne changerait rien pour eux.

6. Le coût de l'absence de schéma : l'adresse sur deux lignes

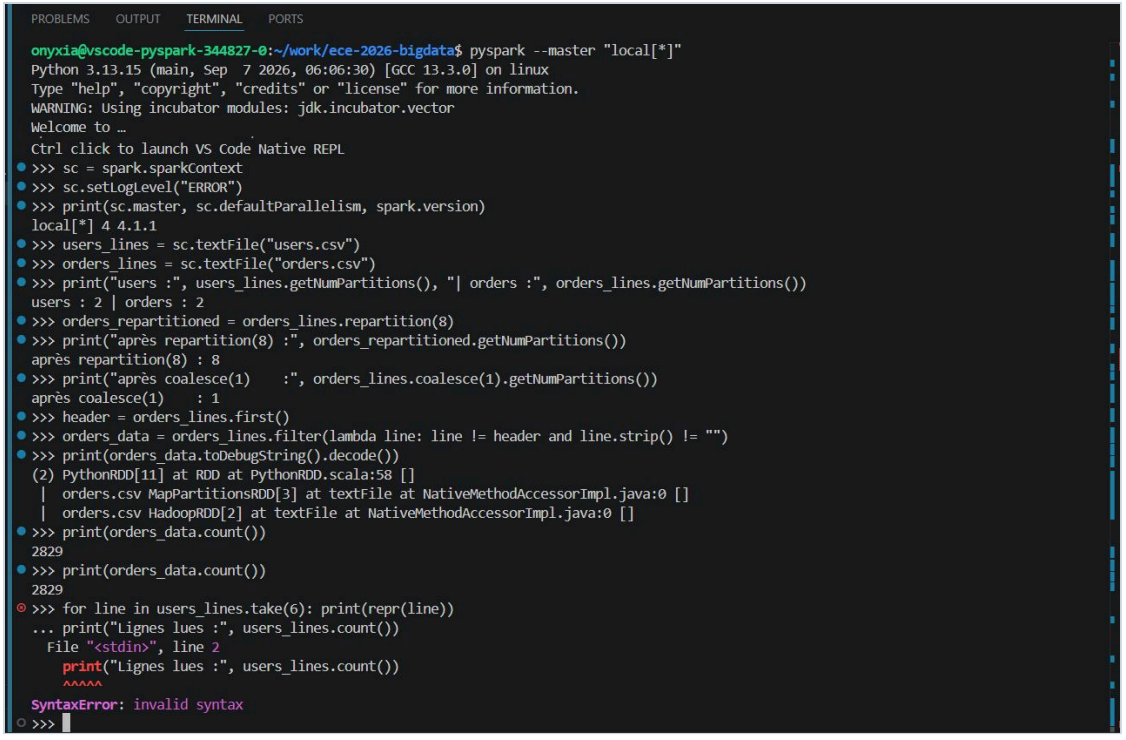


Figure 3 – Première tentative : une boucle for collée avec l'instruction suivante provoque une SyntaxError (difficulté ①)

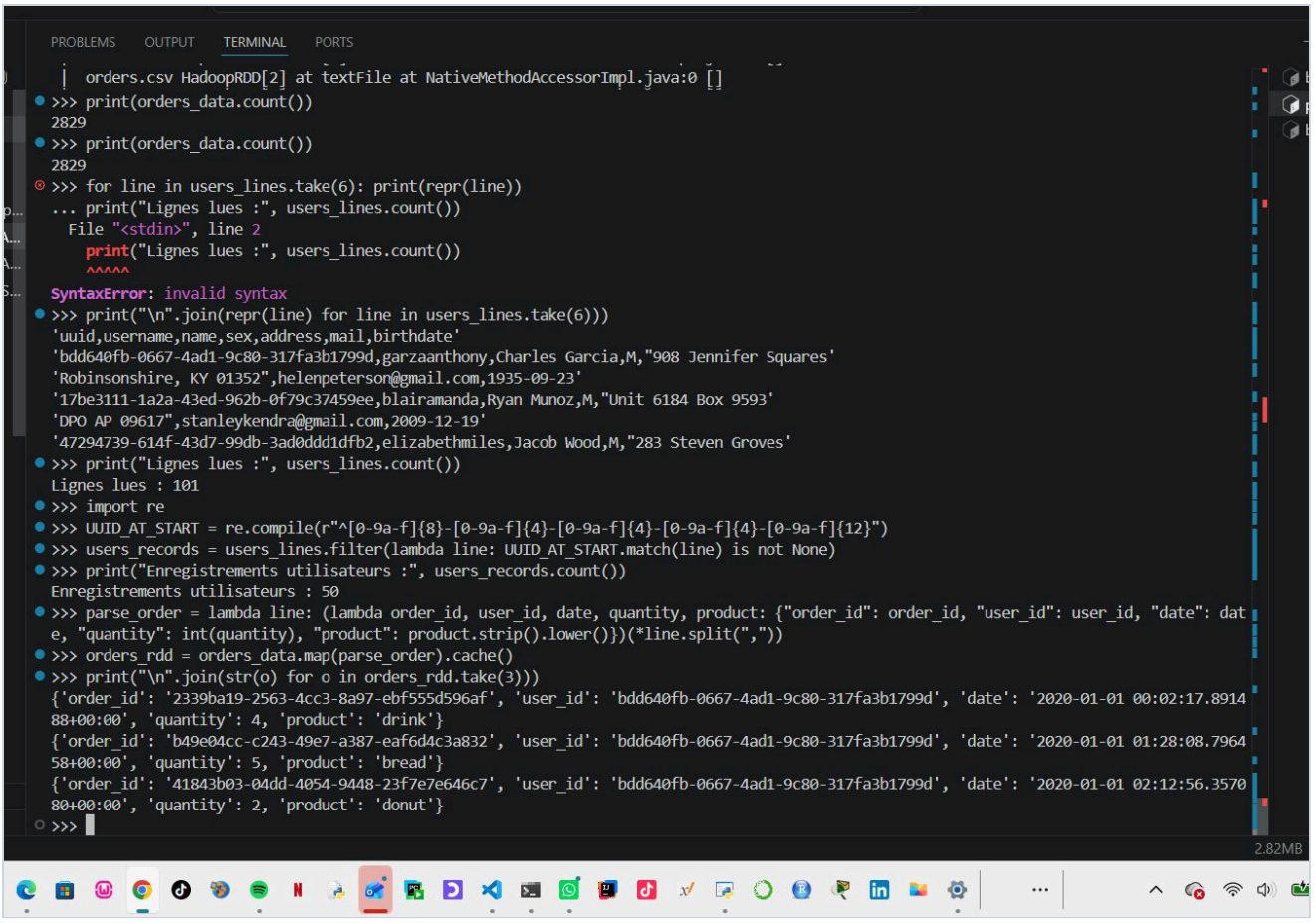


Figure 4 – textFile() coupe chaque adresse en deux : 101 lignes lues ; le filtre sur l'UUID ramène à 50 enregistrements ; parsing des commandes

Chaque enregistrement utilisateur est coupé au milieu de l'adresse : la ligne `'bdd640fb-…,M,"908 Jennifer Squares'` s'arrête à la rue, et la suivante, `'Robinsonshire, KY 01352",helenpeterson@…'`, n'a pas d'UUID et ne peut pas être rattachée à son utilisateur. On lit donc **101 lignes** pour 50 utilisateurs. Le filtre sur l'UUID en tête de ligne ramène à **50 enregistrements**, mais il ne fait qu'écarter les fragments : tout champ issu de l'adresse (ville, État, code postal) reste inutilisable depuis ce RDD.

Q5. Pourquoi spark.read.csv(multiLine=True) n'a-t-il pas ce problème, alors que sc.textFile() l'a ?

`textFile()` est un lecteur **orienté ligne** : il coupe sur chaque `\n`, sans connaître la syntaxe CSV, et traite donc le retour à la ligne entre guillemets comme une fin d'enregistrement. Le lecteur CSV de Spark, avec `multiLine=True`, **analyse la syntaxe CSV** : il suit l'ouverture et la fermeture des guillemets, et sait qu'un `\n` entre guillemets fait partie de la valeur. En contrepartie, il lit chaque fichier d'un seul bloc, car il ne peut pas le découper au hasard.

Q6. De manière générale, que doit garantir un format pour qu'un lecteur ligne par ligne comme textFile() puisse l'utiliser sans risque ?

Un enregistrement par ligne, et jamais de saut de ligne à l'intérieur d'un enregistrement : le `\n` doit servir uniquement de séparateur. C'est le cas du **JSON Lines** (les retours à la ligne dans les valeurs sont échappés), des fichiers de logs, ou d'un CSV sans retour à la ligne dans ses valeurs. Cette propriété permet aussi de **découper** le fichier n'importe où : il suffit d'avancer jusqu'au `\n` suivant pour retrouver le début d'un enregistrement.

7. Parsing manuel des commandes

```
parse_order = lambda line: (lambda order_id, user_id, date, quantity, product: {
    "order_id": order_id, "user_id": user_id, "date": date,
    "quantity": int(quantity), "product": product.strip().lower()})(*line.split(","))
orders_rdd = orders_data.map(parse_order).cache()
```

Chaque commande devient un dictionnaire (figure 4). Seule la quantité est convertie en entier : la date reste une **chaîne de caractères**. Rien ne valide le nombre de champs, le format de la date ni la quantité au moment de la définition : une ligne mal formée ne fera échouer le job qu'à la première action qui l'atteint. Le RDD est mis en cache, car les sections suivantes le réutilisent.

Q7. Que se passerait-il si une ligne de orders.csv contenait une virgule de trop dans product ?

`line.split(",")` produirait 6 champs au lieu de 5, et l'affectation à 5 variables lèverait une erreur (`ValueError`, ou `TypeError` avec la version en `lambda`). Mais pas immédiatement : `map` étant paresseux, la cellule s'exécuterait sans erreur. C'est la **première action** touchant cette ligne qui ferait **échouer tout le job**, après plusieurs tentatives de la tâche, avec une trace d'erreur noyée dans la pile Java et sans indication claire de la ligne en cause. Le découpage naïf ne gère pas non plus une virgule placée entre guillemets, pourtant valide en CSV.

Q8. Où l'API DataFrame effectue-t-elle la vérification équivalente, et quand ?

Dans le **lecteur CSV** (`spark.read.csv`), au moment de la lecture. Le **schéma**, inféré ou déclaré, fixe le nombre et le type des colonnes, et le lecteur gère les guillemets. Une ligne mal formée est traitée selon l'option `mode` : `PERMISSIVE` (champs invalides à `NULL`, ligne brute conservable dans `columnNameOfCorruptRecord`), `DROPMALFORMED` (ligne ignorée) ou `FAILFAST` (arrêt immédiat, avec la ligne en cause). La structure est connue dès la définition du DataFrame, et la validation suit une règle déclarée, et non une erreur Python inattendue au milieu d'un job.

8. RDD clé-valeur : reduceByKey et groupByKey

```
user_quantity = orders_rdd.map(lambda o: (o["user_id"], o["quantity"])).reduceByKey(add)
user_quantity_grouped = (orders_rdd.map(lambda o: (o["user_id"], o["quantity"]))
                         .groupByKey().mapValues(sum))
```

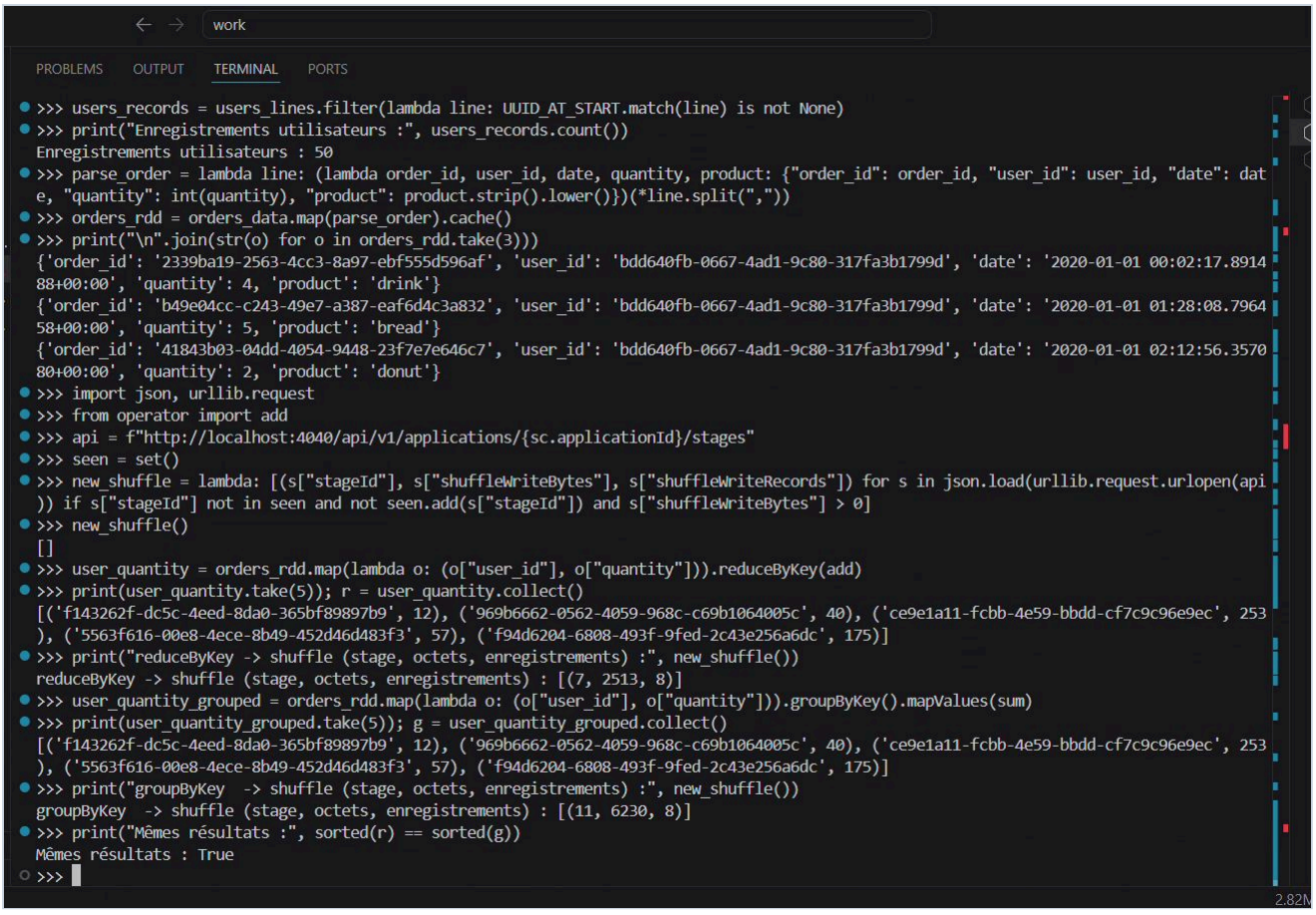


Figure 5 – Mêmes totaux par utilisateur avec les deux méthodes ; shuffle de 2 513 octets avec reduceByKey contre 6 230 avec groupByKey

Méthode	Octets écrits au shuffle	Lots sérialisés
`reduceByKey`	**2 513**	8
`groupByKey`	**6 230** (≈ 2,5×)	8

Les deux méthodes donnent exactement les mêmes résultats, mais `groupByKey` échange 2,5 fois plus de données, même sur ce petit jeu de données. Bien que `take(5)` et `collect()` aient été appelés, un seul stage de shuffle apparaît par méthode : le second job **réutilise les fichiers de shuffle** déjà écrits (stage *skipped*).

Q9. reduceByKey combine les valeurs dans chaque partition avant le shuffle. Que transfère groupByKey à la place, et pourquoi est-ce plus coûteux ?

`groupByKey` transfère **chaque couple** `(user_id, quantité)`, soit les 2829 valeurs. `reduceByKey` additionne d'abord dans chaque partition (*map-side combine*) et n'envoie qu'un total partiel par utilisateur et par partition, au plus 100 valeurs. `groupByKey` coûte donc plus en **réseau et disque**

(plus d'octets à sérialiser, écrire et transférer), en **mémoire** (toutes les valeurs d'une clé sont rassemblées dans une liste, ce qui peut saturer un exécuteur pour une clé fréquente) et en **CPU** (la somme est faite après le shuffle sur des listes complètes).

Q10. Pour quelles agrégations groupByKey serait-il inévitable ?

Quand le calcul a besoin de **toutes les valeurs d'une clé à la fois**, sans décomposition en résultats partiels combinables : la **médiane** ou un percentile exact, le **tri des valeurs** d'une clé (commandes d'un client dans l'ordre chronologique, reconstitution de sessions), la liste complète des valeurs, ou un traitement qui consomme la liste entière (entraîner un modèle par client). Une agrégation associative et commutative (somme, min, max, comptage, moyenne via un couple somme/nombre) doit toujours passer par `reduceByKey` ou `aggregateByKey`.

9. Dédoublonnage manuel

```
most_recent = lambda a, b: a if a["date"] > b["date"] else b
orders_dedup = (orders_rdd.map(lambda o: (o["order_id"], o))
                .reduceByKey(most_recent).values())
```

Résultat : 2829 commandes avant comme après, puisque le fichier ne contient pas de doublons (figure 6). Le shuffle pèse cette fois **186 001 octets** : ce sont les **dictionnaires complets** des commandes qui sont échangés, et non plus de simples entiers.

Q11. Dans la fonction de fenêtre, partitionBy() regroupe toutes les lignes d'un même order_id sur la même partition. Qu'est-ce qui joue ce rôle ici ?

`reduceByKey`. Le `map` transforme chaque commande en couple `(order_id, commande)`, puis `reduceByKey` fait un **shuffle par hachage de la clé** : toutes les versions d'un même `order_id` arrivent sur la même partition, où `most_recent` est appliquée, d'abord localement dans chaque partition, puis après le shuffle.

Q12. Quelle version exprime le plus directement l'intention : la fonction de fenêtre ou reduceByKey avec une fonction de combinaison ?

La **fonction de fenêtre** : `row_number() OVER (PARTITION BY order_id ORDER BY ordered_at DESC) = 1` se lit comme « garder la ligne la plus récente de chaque commande ». La version RDD oblige à reconstruire cette intention à partir d'une clé construite à la main, d'une fonction de comparaison et d'un `values()` ; elle compare en plus des **dates en texte**, ce qui ne fonctionne que grâce au format ISO. En contrepartie, elle est efficace : `reduceByKey` combine avant le shuffle, alors que la fenêtre trie toutes les lignes, en partie compensée par l'optimisation `WindowGroupLimit`.

10. Jointure des commandes et des utilisateurs

```
users_kv = users_records.map(lambda line: tuple(line.split(",")[:2]))   # (uuid, username)
orders_kv = orders_rdd.map(lambda o: (o["user_id"], o))
enriched = orders_kv.join(users_kv)
```

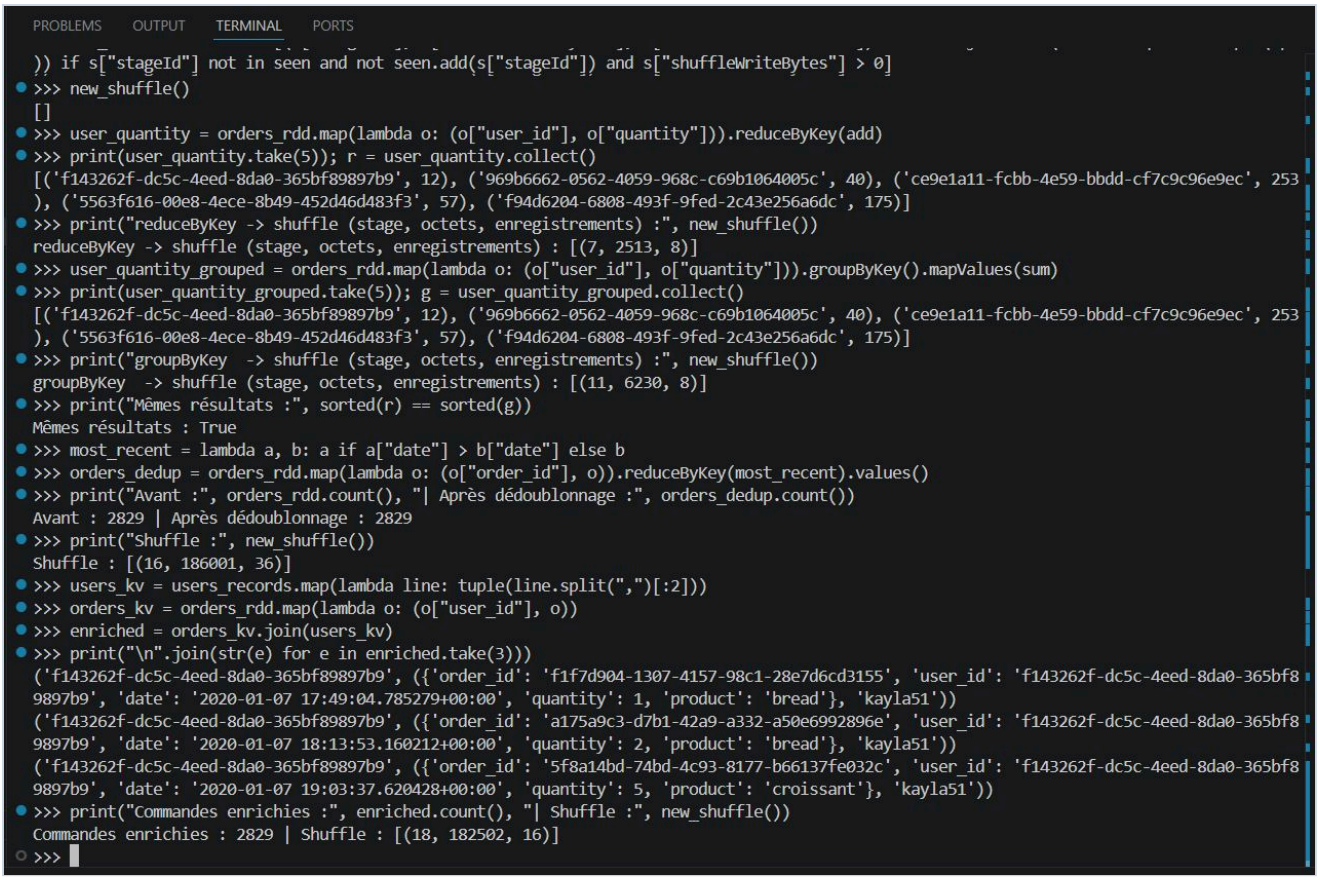


Figure 6 – Dédoublonnage (2829 commandes, shuffle de 186 Ko) et jointure : 2829 commandes enrichies avec le username, shuffle de 182 Ko en 16 lots

Chaque élément de `enriched` a la forme `(user_id, (commande, username))`. Les 2829 commandes trouvent leur utilisateur. Le shuffle compte **16 lots au lieu de 8** : la jointure redistribue **les deux RDD**.

Q13. join() est une transformation large sur les deux RDD. Comment orders_kv et users_kv devraient-ils être partitionnés pour que Spark évite le shuffle ?

Ils devraient être **co-partitionnés** : partitionnés par la même clé (`user_id`), avec le **même partitionneur** et le **même nombre de partitions**, par exemple `partitionBy(4)` sur les deux puis `cache()` pour conserver ce partitionnement. Les lignes d'un utilisateur se trouvent alors dans la partition de même numéro des deux côtés, et la jointure se fait partition par partition, sans échange réseau. Le shuffle n'est payé qu'une fois, au `partitionBy`. Avec les DataFrames, on obtient un effet comparable avec le *bucketing*, ou avec une **jointure par diffusion** quand une table est petite, comme `users`.

Q14. Remplacez join() par leftOuterJoin(). Que change-t-il si un user_id des commandes n'a pas d'utilisateur correspondant ?

Avec `join()`, jointure interne, la commande **disparaît du résultat** sans avertissement. Avec `leftOuterJoin()`, elle est **conservée** sous la forme `(user_id, (commande, None))`, ce qui permet de repérer et compter les commandes orphelines (exercice 2).

Q15. enriched n'est encore ni filtré ni agrégé. Quel est le risque d'appeler .collect() dessus, sur un jeu de données bien plus gros ?

`collect()` **rapatrie tout le RDD dans le driver**, un seul processus à la mémoire limitée. Sur un gros volume, le driver manque de mémoire (`OutOfMemoryError`) ou dépasse `spark.driver.maxResultSize`, et la session plante. Le volume est en plus gonflé par la jointure, chaque commande embarquant une copie des informations de son utilisateur. Il faut filtrer ou agréger d'abord, inspecter avec `take(n)` ou `count()`, ou écrire le résultat sur le stockage.

11. Exercices

Exercice 1 – `aggregateByKey` en une seule passe

```
stats = (orders_rdd.map(lambda o: (o["user_id"], o["quantity"]))
         .aggregateByKey((0, 0),
                         lambda acc, q: (acc[0] + 1, acc[1] + q),          # dans une partition
                         lambda a, b: (a[0] + b[0], a[1] + b[1])))        # entre partitions
avg_qty = stats.mapValues(lambda s: (s[0], s[1], round(s[1] / s[0], 2)))
```

`aggregateByKey` prend une valeur initiale, une fonction qui intègre une valeur dans l'accumulateur d'une partition, et une fonction qui fusionne les accumulateurs entre partitions. On obtient en **une seule passe** le nombre de commandes et la quantité totale de chaque utilisateur, puis la moyenne (figure 7). Par exemple, un client a passé 85 commandes pour 253 articles, soit 2,98 en moyenne. Les totaux sont identiques à ceux de `reduceByKey`. Une moyenne ne peut pas être calculée directement par `reduceByKey` (une moyenne de moyennes partielles serait fausse) : le couple `(nombre, somme)` la rend décomposable.

Exercice 2 – `leftOuterJoin` avec un utilisateur inconnu

```
fake = sc.parallelize([{"order_id": "fake-order-1",
                        "user_id": "00000000-0000-0000-0000-000000000000", ...}}])
left = orders_rdd.union(fake).map(lambda o: (o["user_id"], o)).leftOuterJoin(users_kv)
orphans = left.filter(lambda kv: kv[1][1] is None)
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS

>>> print("Shuffle :", new_shuffle())
Shuffle : [(16, 186001, 36)]
>>> users_kv = users_records.map(lambda line: tuple(line.split(",")[:2]))
>>> orders_kv = orders_rdd.map(lambda o: (o["user_id"], o))
>>> enriched = orders_kv.join(users_kv)
>>> print("\n".join(str(e) for e in enriched.take(3)))
('f143262f-dc5c-4eed-8da0-365bf89897b9', ({'order_id': 'f1f7d904-1307-4157-98c1-28e7d6cd3155', 'user_id': 'f143262f-dc5c-4eed-8da0-365bf8
9897b9', 'date': '2020-01-07 17:49:04.785279+00:00', 'quantity': 1, 'product': 'bread'}, 'kayla51'))
('f143262f-dc5c-4eed-8da0-365bf89897b9', ({'order_id': 'a175a9c3-d7b1-42a9-a332-a50e6992896e', 'user_id': 'f143262f-dc5c-4eed-8da0-365bf8
9897b9', 'date': '2020-01-07 18:13:53.160212+00:00', 'quantity': 2, 'product': 'bread'}, 'kayla51'))
('f143262f-dc5c-4eed-8da0-365bf89897b9', ({'order_id': '5f8a14bd-74bd-4c93-8177-b66137fe032c', 'user_id': 'f143262f-dc5c-4eed-8da0-365bf8
9897b9', 'date': '2020-01-07 19:03:37.620428+00:00', 'quantity': 5, 'product': 'croissant'}, 'kayla51'))
>>> print("Commandes enrichies :", enriched.count(), "| Shuffle :", new_shuffle())
Commandes enrichies : 2829 | Shuffle : [(18, 182502, 16)]
>>> stats = orders_rdd.map(lambda o: (o["user_id"], o["quantity"])).aggregateByKey((0, 0), lambda acc, q: (acc[0] + 1, acc[1] + q), lambd
a a, b: (a[0] + b[0], a[1] + b[1]))
>>> avg_qty = stats.mapValues(lambda s: (s[0], s[1], round(s[1] / s[0], 2)))
>>> print("\n".join(str(x) for x in avg_qty.take(5)))
('f143262f-dc5c-4eed-8da0-365bf89897b9', (4, 12, 3.0))
('969b6662-0562-4059-968c-c69b1064005c', (12, 40, 3.33))
('ce9e1a11-fcbb-4e59-bbdd-cf7c9c96e9ec', (85, 253, 2.98))
('5563f616-00e8-4ece-8b49-452d46d483f3', (21, 57, 2.71))
('f94d6204-6808-493f-9fed-2c43e256a6dc', (49, 175, 3.57))
>>> print("Totaux identiques à reduceByKey :", sorted(stats.mapValues(lambda s: s[1]).collect()) == sorted(r))
Totaux identiques à reduceByKey : True
>>> fake = sc.parallelize([{"order_id": "fake-order-1", "user_id": "00000000-0000-0000-0000-000000000000", "date": "2020-05-01 00:00:00+0
0:00", "quantity": 1, "product": "bread"}])
>>> orders_with_fake = orders_rdd.union(fake).map(lambda o: (o["user_id"], o))
>>> left = orders_with_fake.leftOuterJoin(users_kv)
>>> orphans = left.filter(lambda kv: kv[1][1] is None)
>>> print("join interne :", orders_with_fake.join(users_kv).count(), "| leftOuterJoin :", left.count(), "| sans utilisateur :", orphans.c
ount())
join interne : 2829 | leftOuterJoin : 2830 | sans utilisateur : 1
>>> print(orphans.collect())
[('00000000-0000-0000-0000-000000000000', ({'order_id': 'fake-order-1', 'user_id': '00000000-0000-0000-0000-000000000000', 'date': '2020-
05-01 00:00:00+00:00', 'quantity': 1, 'product': 'bread'}, None))]
```

Figure 7 – Exercice 1 : nombre de commandes, quantité et moyenne par utilisateur ; exercice 2 : la jointure interne perd la fausse commande, le leftOuterJoin la conserve avec None

La jointure interne renvoie **2829** lignes et perd silencieusement la fausse commande ; le `leftOuterJoin` en renvoie **2830**, dont **1 sans utilisateur**, associée à `None`. C'est l'équivalent RDD du test `relationships` du lab dbt.

Exercice 3 – La même agrégation avec l'API DataFrame

```
orders_df = spark.read.csv("orders.csv", header=True, inferSchema=True)
user_quantity_df = orders_df.groupBy("user_uuid").sum("quantity")
```

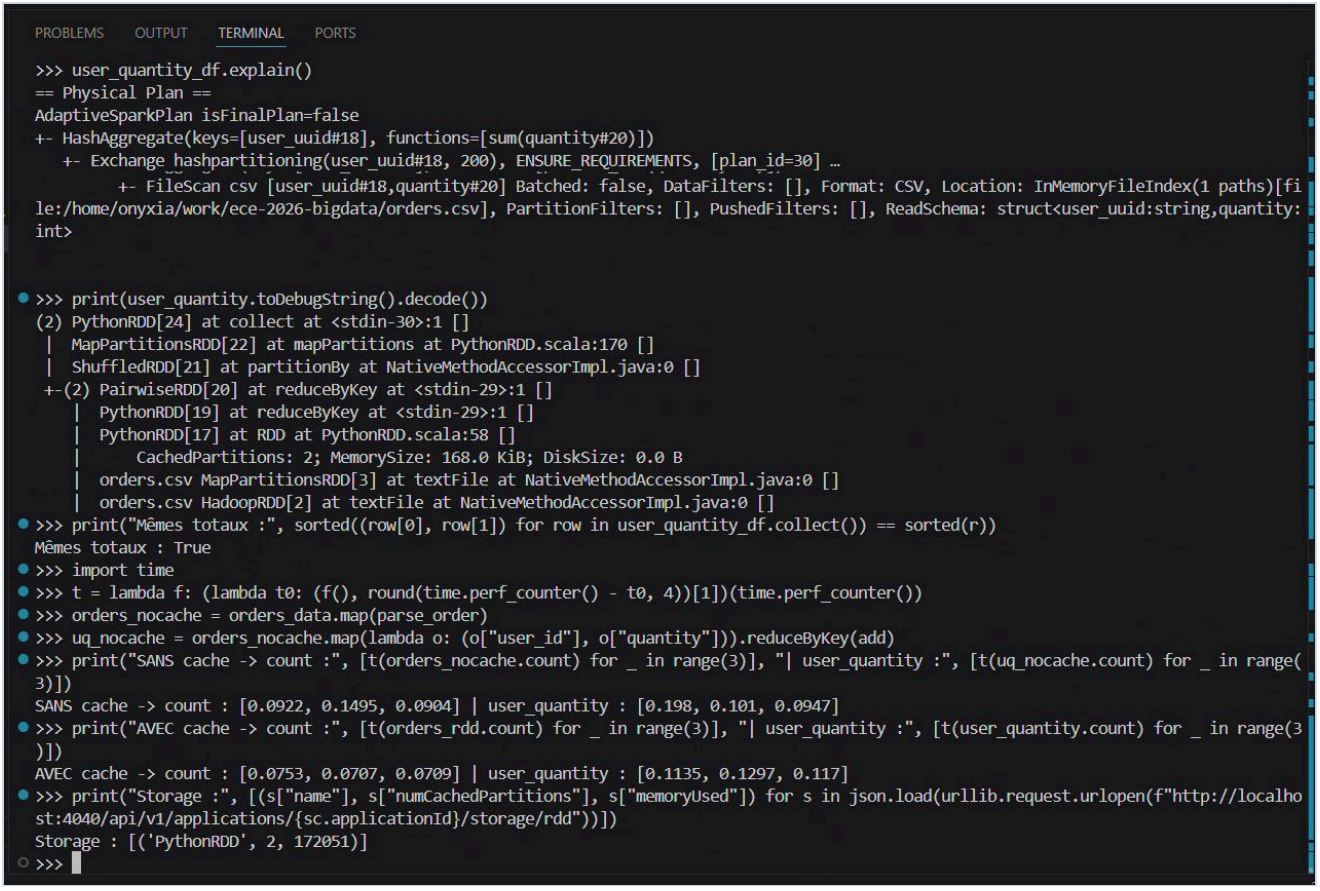


Figure 8 – Exercice 3 : plan physique du DataFrame et lignage du RDD, mêmes totaux ; exercice 4 : temps avec et sans cache, et RDD en cache

	RDD	DataFrame
Code	Lecture, filtre de l'en-tête, fonction de parsing, `map` vers des couples, `reduceByKey`	**2 lignes** : lecture avec schéma, puis `groupBy().sum()`
Description de l'exécution	`toDebugString` : `PythonRDD`, `ShuffledRDD`, `PairwiseRDD`, `HadoopRDD`	`explain()` : `FileScan csv`, `Exchange hashpartitioning`, `HashAggregate`
Colonnes lues	Lignes entières, découpées en Python	**Seulement** `user_uuid` et `quantity` (`ReadSchema`)
Exécution	Fonctions Python, données sérialisées entre JVM et Python	Entièrement dans la JVM, plan optimisé par Catalyst

Les totaux sont identiques. La version DataFrame est bien plus courte et déclarative : elle dit **quoi** calculer, et Catalyst choisit **comment**. Son plan montre un `HashAggregate` après l'`Exchange`,

avec une agrégation partielle avant le shuffle, l'équivalent automatique du *map-side combine* de `reduceByKey`. Il ne lit que les deux colonnes utiles, alors que le RDD transporte des lignes entières et les découpe en Python. Le lignage du RDD n'est qu'une chaîne de transformations opaques, que Spark ne peut ni réordonner ni optimiser.

Exercice 4 – Effet de `.cache()`

Mesure (3 essais, en s)	Sans cache	Avec cache
`orders_rdd.count()`	0,092 · 0,150 · 0,090	**0,075 · 0,071 · 0,071**
`user_quantity.count()`	0,198 · 0,101 · 0,095	0,114 · 0,130 · 0,117

Le comptage des commandes est environ **20 % plus rapide** avec le cache, et plus stable : le fichier n'est plus relu ni parsé en Python. L'onglet Storage, lu par l'API, montre le RDD en cache (`PythonRDD`, **2 partitions**, environ **168 Kio** en mémoire), et le lignage de l'exercice 3 indique `CachedPartitions: 2; MemorySize: 168.0 KiB`. Pour `user_quantity`, l'écart n'apparaît pas : le RDD agrégé réutilise ses fichiers de shuffle déjà écrits, dans les deux versions. Sur un fichier aussi petit, les temps sont surtout dominés par les coûts fixes de planification et de lancement des tâches ; le gain du cache deviendrait important sur un fichier volumineux ou un parsing coûteux.

12. Synthèse et conclusion

Étape	Résultat
Partitions	2 par défaut, 8 après `repartition`, 1 après `coalesce`
Évaluation paresseuse	Lignage `PythonRDD → MapPartitionsRDD → HadoopRDD` ; chaque action relance le calcul
Adresse sur deux lignes	101 lignes lues pour 50 utilisateurs ; 50 après filtre sur l'UUID
`reduceByKey` / `groupByKey`	Mêmes totaux ; shuffle de 2 513 contre 6 230 octets
Dédoublonnage	2829 → 2829 commandes, shuffle de 186 Ko
Jointure	2829 commandes enrichies, shuffle des deux RDD (182 Ko)
Exercices	Moyenne par `aggregateByKey` ; 1 commande orpheline détectée par `leftOuterJoin` ; DataFrame en 2 lignes avec les mêmes totaux ; cache environ 20 % plus rapide

Ce lab a reconstruit à la main ce que l'API DataFrame et Spark SQL fournissent automatiquement : la lecture correcte d'un CSV dont les valeurs contiennent des retours à la ligne, la gestion des types et des erreurs de format, le choix de l'agrégation partielle avant le shuffle, et la lecture des seules colonnes utiles. Avec les RDD, chacun de ces points est à la charge du développeur, et une erreur ne se manifeste souvent qu'à l'exécution d'une action.

Les RDD restent utiles pour comprendre le fonctionnement de Spark (partitions, lignage, transformations étroites et larges, shuffle), ou pour traiter des données sans structure tabulaire. Mais pour des données structurées, l'API DataFrame est plus concise, plus sûre et plus performante, car le moteur peut optimiser un plan qu'il comprend, alors que des fonctions Python lui restent opaques.